

Got it — you want to **add a timeout** to an existing

CancellationToken

in C#, using a **linked token source**, but you also want to **detect if the cancellation came specifically from the timeout** rather than from the original token.

Here's a **complete, runnable** example that does exactly that:

```
using System;
using System.Threading;
using System.Threading.Tasks;

class Program
{
    static async Task Main()
    {
        // Simulate an existing cancellation token (e.g., from caller)
        using var existingCts = new CancellationTokenSource();

        // Create a timeout CTS
        TimeSpan timeout = TimeSpan.FromSeconds(3);
        using var timeoutCts = new CancellationTokenSource(timeout);

        // Link both tokens
        using var linkedCts = CancellationTokenSource.CreateLinkedTokenSource(
            existingCts.Token, timeoutCts.Token);

        try
```

```
{
    Console.WriteLine("Starting operation...");
    await DoWorkAsync(linkedCts.Token);
    Console.WriteLine("Operation completed successfully.");
}
catch (OperationCanceledException) when (timeoutCts.IsCancellationRequested &&
!existingCts.IsCancellationRequested)
{
    Console.WriteLine("Operation canceled due to TIMEOUT.");
}
catch (OperationCanceledException) when (existingCts.IsCancellationRequested)
{
    Console.WriteLine("Operation canceled due to ORIGINAL token.");
}
}

static async Task DoWorkAsync(Cancellation_token token)
{
    // Simulate long-running work
    for (int i = 0; i < 10; i++)
    {
        token.ThrowIfCancellationRequested();
        Console.WriteLine($"Working... {i + 1}");
        await Task.Delay(1000, token); // cancellable delay
    }
}
}
```

How it works

- `existingCts`
 - represents the original cancellation token from the caller.
- `timeoutCts`
 - automatically cancels after the given timeout.
- `linkedCts`
 - combines both tokens so the async operation can be canceled by either.
- **Catch filters:**
 - `when (timeoutCts.IsCancellationRequested && !existingCts.IsCancellationRequested)`
 - timeout only.
 - `when (existingCts.IsCancellationRequested)`
 - original token only.

Example Output

If the work takes longer than 3 seconds:

```
Starting operation...  
Working... 1
```

```
Working... 2  
Working... 3  
Operation canceled due to TIMEOUT.
```

If you cancel

```
existingCts
```

manually before timeout:

```
Starting operation...  
Working... 1  
Operation canceled due to ORIGINAL token.
```

If you want, I can also give you a **version that returns a custom result instead of throwing** so you don't need try/catch for timeout detection.
Do you want me to prepare that?